- JavaScript
  - 基本数据类型之symbol、bigint
  - foreEach、for……of、for……in的区别
  - 数组常用的方法？哪些改变原数组？哪些不改变？
  - 数组乱序
  - 构造函数
  - 原型链
  - 数组去重（set）
  - 预编译
    - 函数声明整体提升
    - 变量声明提升（变量赋值不提升）
  - this指向问题
  - 箭头函数和普通函数的区别
    - 箭头函数无法创建对象，只能当做函数来使用
    - 箭头函数可以同时省略 {} 和 return
    - 箭头函数没有arguments
    - 箭头函数无this，其this值取决于最近外层的非箭头函数
    - 箭头函数的参数不能重复命名
  - 逗号运算符配合log
  - 闭包
  - 高阶函数
    - 一个函数，可以接收函数，或者返回函数
  - JS判断数组中是否包含某个值
    - indexOf(searcElement)
      - 此方法判断数组中是否存在某个值，如果存在，则返回数组元素的下标，否则返回-1
    - includes(searcElement[,fromIndex])
      - 此方法判断数组中是否存在某个值，如果存在返回true，否则返回false
    - find(callback[,thisArg])
      - 返回数组中满足条件的第一个元素的值，如果没有，返回undefined
    - findeIndex(callback[,thisArg])
      - 返回数组中满足条件的第一个元素的下标，如果没有找到，返回-1

```
//a未定义
console.log(typeof (typeof a));
```

- ES6

- let、const、var的区别
  - var、let、const的区别
- var
  - 可重复定义
  - 变量声明提升
  - 挂载到window对象上
  - 作用域只有全局或函数
- let
  - 不可重复定义
  - 无变量声明提升
  - 不会挂载到window上
  - 会和大括号形成块级作用域
- const
  - 与let类似，但是声明必须赋值，且赋值后不可修改

- Array arr.flat([depth])：扁平化多维数组
  - depth：指定提取的深度，默认值为1（如果数组较深可传Infinity）

```
1、临时死区
let a = 10;
{
    let a = 20;  //放下边呢?
    console.log(a);
}

{
    var a = 5;
    let b = 6;
}
console.log(a);
console.log(b);

2、求平均值
function avg1(...arg) {
    let sum = 0;
    arg.forEach(function (value) {
        sum += value;
    })
    return sum/arg.length;
```

```
}

3、reduce()
/*
Array.reduce( (did,cur,index,arr) => {},init);
* did：累计器（操作后的结果）
* cur：当前值
* index：当前索引，如果有init索引从0开始，没有则从1开始
* arr：初始值（可以是对象、数组等）
* */

//数组去重
let arr6 = [1,2,2,3,3,3];
let sum6 = arr6.reduce((did,cur) => {
    if (!did.includes(cur)){
        did.push(cur);
    }
    return did;
},[]);
console.log(sum6);
```

- Promise的其他方法
  - all：所有异步操作都成功后再走then()，可以拿到全部结果
  - any：　　接收一个 Promise 可迭代对象，只要其中的一个成功（必须是成功），就返回成功的结果（只关注最先成功的结果并且返回，忽略失败）
  - race：　　接收一个 Promise 可迭代对象，只要其中的一个完成（成功或失败），就返回完成的结果（会关注最先完成的结果并且返回，成败均可）
  - finally：传入一个函数，不论成功失败都会执行，类似于switch中的default

```
const p0 = () => new Promise((resolve, reject) => setTimeout(reject, 300, "p0"));
const p1 = new Promise((resolve, reject) => setTimeout(resolve, 500, "p1"));
const p2 = new Promise((resolve, reject) => setTimeout(resolve, 1000, "p2"));

Promise.all([p1, p2]).then(res => console.log(res));  //["p1", "p2"]
Promise.any([p0(), p1, p2]).then(res => console.log(res));  //"p1"   （最先成功）
Promise.race([p0(), p1, p2]).then(res => console.log(res)).catch(res => console.log(res));  //"p0"   （最先完成：失败）
p0().catch(res => console.log(res)).finally(() => console.log("finally"));  //p0 finally
```

- Promise的3个状态
  - pending：等待/进行中
  - resolved：操作成功
  - rejected：操作失败

- 手写
  - map、push、pop、filter、reduce、call、apply

- EventLoop

```
Promise.resolve().then(()=>{
  console.log('Promise1')
  setTimeout(()=>{
    console.log('setTimeout2')
  },0)
});
setTimeout(()=>{
  console.log('setTimeout1')
  Promise.resolve().then(()=>{
    console.log('Promise2')
  })
},0);
console.log('start');

// start
// Promise1
// setTimeout1
// Promise2
// setTimeout2

Promise.resolve().then(() => {
    console.log("Promise1");
    setTimeout(() => {
        console.log("setTimeout2");
    }, 0);
});
setTimeout(() => {
    console.log("setTimeout1");
```

```
    Promise.resolve().then(() => {
        console.log("Promise2");
    });
}, 5);
console.log("start");
-----------------------------------------------------
Promise.resolve().then(() => {
    console.log("Promise1");
    setTimeout(() => {
        console.log("setTimeout2");
    }, 0);
});
setTimeout(() => {
    console.log("setTimeout1");
    Promise.resolve().then(() => {
        console.log("Promise2");
    });
}, 10);
console.log("start");
```

- nth-of-type等

```
div:not(.test){  //属性名不是test的div  }
div:not([class="test"]){  //属性名不是test的div  }
div:not([class]){  //没有属性名的div  }
li:not(:last-of-type){  //除了最后一个li  }
```

- flex复合属性
  - flex-grow、flex-shrink、flex-basis
    - 默认值：0 1 auto，后2个属性值可选
  - flex-grow：用数值来定义扩展比率
    - 剩余的空间会按照每个子元素中的flex-grow属性值来分配
  - flex-shrink：用数值来定义收缩比率
    - 按照子元素content宽度与收缩比率的加权值来压缩，大的压缩多，小的压缩少
  - flex-basis
    - 仅设定此属性时，元素宽度为此值，但可以被内容无限撑大

        - 与width同时设定时，元素宽度为此值，但可以被内容撑大不超过width（如果width比flex-basis小，则元素宽度固定为flex-basis不变）
- 画出三角形

```
div {
    height: 0;
    width: 0;
    border: 100px solid transparent;
    border-top-color: #f00;
}
```

- 描述一下样子

```
.wrapper {
    width: 1000px;
    height: 1000px;
    display: flex;
    border: 1px solid #f00;
}

.wrapper div {
    width: 100px;
    border: 1px solid #0a8cd2;
}

<div class="wrapper">
    <div></div>
    <div></div>
    <div></div>
    <div></div>
    <div></div>
</div>
```

- BFC的渲染规则、触发条件、应用
    - 渲染规则
        - BFC的区域不会与浮动元素的box重叠
        - BFC是一个独立的容器，外面的元素不会影响里面的元素
        - BFC垂直方向边距重叠

        - 计算BFC高度的时候浮动元素也会参与计算
    - 触发条件
        - float：left / right
        - overflow：非默认值
        - position：absolute / fixed
        - 根元素
        - display：inline-block / flex
    - 应用
        - 防止父元素高度塌陷
        - 避免外边距重叠

```
<div class="one"></div>
<div class="two">
    <div class="one"></div>
</div>
<div class="one"></div>

body {
    background-color: #0ff;
}

.one {
    width: 200px;
    height: 200px;
    background-color: #0f0;
    margin: 20px;
}

.two {
    /*overflow: hidden;*/
}
```

- 浏览器画图方式
    - canvas：点阵图，缩放失真，适合图形数量非常大的图表
    - svg：矢量图，缩放不失真，适合图形数量较少的图表
- 虚拟DOM
    - 操作DOM是很慢的，如果我们把一个简单的div元素属性打印出来，会看到很多属性

- 相对于DOM对象，原生的JavaScript对象处理起来更快，而且更简单。DOM树上的结构、属性信息我们都可以很容易用JavaScript对象表示出来
- 步骤
  - 用JavaScript对象结构表示DOM树的结构；然后用这个数构建一个真正的DOM树，插到文档当中
  - 当状态变更的时候，重新构造一颗新的对象树。然后用新的树和旧的树进行比较，记录两棵树的差异
  - 把2所记录的差异应用到步骤1所构建的真正的DOM树上，视图就更新了
- 原理
  - 虚拟DOM本质上就是在JS和DOM之间做了一个缓存。可以类比CPU和硬盘，既然硬盘这么慢，我们就在它们之间加一个缓存
  - 既然DOM这么慢，我们就在JS和DOM之间加个缓存，JS只操作虚拟DOM，最后的时候再把变更写入DOM
- Diff算法的说明
  - 如果两棵树的根元素类型不同，会销毁旧树，创建新树
  - 对于类型相同的DOM元素，会对比两者属性是否相同，只更新不同的属性

- 内置组件 - keep-alive
  - 切换组件的时候，组件中的内容会保留下来，不会每次都清空

- React

```
export default function() {
  const [num, setNum] = useState(0);
  const click = () => {
      setNum(num + 1);  //每次加1，更新1次
      setNum(num + 1);
      setNum(num + 1);
    //setNum(num => num + 1);  //每次加3，更新1次
    //setNum(num => num + 1);
    //setNum(num => num + 1);
  };

  useEffect(() => {
    console.log("更新了");
  });

  return (
```

```
    <>
      <p>{num}</p>
      <button onClick={click}>按钮</button>
    </>
  );
}
```

- 字符串JSX的处理方式

```
如果后端给你返回的数据是字符串类型的JSX,
譬如: let jsx = '<div>呵呵</div>';
将其插入到节点中, 有哪些方式?

let jsx = '<div>呵呵</div>';
1、使用 ref
<div ref={ node => node.innerHTML = jsx }/>
2、使用 dangerouslySetInnerHTML
<div dangerouslySetInnerHTML = {{__html:jsx}}/>
```